

PUP Mathematics Gradient 2026
Python Workshop: Initialize

Engr. Jaydee N. Lucero
May 9, 2026

Hi! I'm Jaydee.

- * *Senior Structural Engineer I* (2021-present), Abinales Associates Engineers + Consultants
- * *Part-time Review Instructor* (2019-present), Review Innovations
- * *Associate Member* (2022-present), Association of Structural Engineers of the Philippines, Inc.
- * *MS Civil Engineering (Structural Engineering) student* (2024-present), University of the Philippines Diliman
- * Pythonista since August 2024



Outline

1. Why Python?
2. The *numpy* library
3. The *scipy*, *sympy* and *matplotlib* libraries
4. The *pandas* library
5. A famous math example
6. Next steps

QR Codes

Live codes for this talk



Python 3.14 cheat sheet



Part 1

Why Python?

Why Python?

Python website

- * Python is powerful.
- * Python is fast.
- * Python plays well with others.
- * Python runs everywhere.
- * Python is friendly.
- * Python is easy to learn.
- * Python is open[-source].



Why Python?

Python website

- * Python is powerful.
- * Python is fast.
- * Python plays well with others.
- * Python runs everywhere.
- * Python is friendly.
- * Python is easy to learn.
- * Python is open[-source].

A C program to determine the maximum value in an array

```
1 #include <stdio.h>
2
3 int main(void) {
4     int arr[] = {3, 1, 6, 4};
5     int n = 4;
6     int max = arr[0];
7     int i;
8
9     for(i = 1; i < n; i++)
10         if(arr[i] > max)
11             max = arr[i];
12
13     printf("Max = %d\n", max);
14     return 0;
15 }
```

Equivalent Python program

```
1 arr = [3, 1, 6, 4]
2 print(max(arr))
```

Why Python?

Python website

- * Python is powerful.
- * Python is fast.
- * Python plays well with others.
- * Python runs everywhere.
- * Python is friendly.
- * Python is easy to learn.
- * Python is open[-source].

TIOBE Index, April 2026

Rank	Language	Rating
1	Python	20.97%
2	C	12.34%
3	C++	8.03%
4	Java	7.79%
5	C#	5.98%
6	Javascript	3.11%
7	VB	3.02%
8	SQL	1.75%
9	R	1.62%

Python is powerful.

Some of the Python libraries that we need in applied mathematics.

Library	Function
numpy	arrays, fast math, linear algebra
scipy	numerical math, stats, regression
sympy	symbolic math
pandas	data extraction, cleaning, analysis
matplotlib	data visualization

Part 2

The numpy library

The numpy library

```
1 import numpy as np
```

for loops and list comprehensions are slow when creating array-like objects.

```
1 import time
2
3 t_start = time.time()
4 result = [x*x2 for x in range(1_000_000)]
5 t_end = time.time()
6 print(round(t_end - t_start, 4))           # 0.1069 seconds
```

numpy arrays can make the same instruction several times faster.

```
1 import time
2 import numpy as np
3
4 t_start = time.time()
5 result = np.arange(1_000_000) * 2
6 t_end = time.time()
7 print(round(t_end - t_start, 4))           # 0.0213 seconds
```

The numpy library

Example 1

Solve the system of linear equations

$$5x + 3y + 2z = 15$$

$$2x - 6y - 7z = -22$$

$$4x - 8y + 3z = 9$$

using

1. matrix inverse method,
2. Cramer's rule, and
3. *numpy*'s own solver function.

The numpy library

Solution to #1

```
1 import numpy as np
2
3 A = [[5, 3, 2], [2, -6, -7], [4, -8, 3]]
4 B = [15, -22, 9]
```

Iterators like lists and tuples can be converted to *numpy* arrays.

```
1 A = np.array(A, dtype='float')
2 B = np.array(B, dtype='float')
```

Check if the system has a unique solution by finding the rank of A.

```
1 A_rank = np.linalg.matrix_rank(A)    # matrix rank
2 print(rf"rank(A) = {A_rank}")        # rank(A) = 3
```

Now, solve the system using $AX = B \rightarrow X = A^{-1} B$.

```
1 A_inv = np.linalg.inv(A)              # matrix inverse
2 X = A_inv @ B                         # matrix multiplication
3 print(rf"X = {X} using matrix inverse method.")
4 # X = [1.40570175  0.68640351  2.95614035] using matrix inverse method.
```

The numpy library

Solution to #2

Copy the coefficients matrix A by value.

```
1 A_x = np.copy(A)
2 A_y = np.copy(A)
3 A_z = np.copy(A)
```

Replace each column in the coefficients matrix with the constants vector.

```
1 print(A)           # [[ 5,  3,  2], [ 2, -6, -7], [4, -8, 3]]
2 A_x[:, 0] = B       # [[15,  3,  2], [-22, -6, -7], [9, -8, 3]]
3 A_y[:, 1] = B       # [[ 5, 15,  2], [ 2, -22, -7], [4,  9, 3]]
4 A_z[:, 2] = B       # [[ 5,  3, 15], [ 2, -6, -22], [4, -8, 9]]
```

Next, calculate the required determinants.

```
1 D = np.linalg.det(A)           # determinant
2 D_x = np.linalg.det(A_x)
3 D_y = np.linalg.det(A_y)
4 D_z = np.linalg.det(A_z)
```

The numpy library

Solution to #2 (continued)

Now, solve the system using $X = D_x/D$, $Y = D_y/D$, $Z = D_z/D$.

```
1 X = np.array([D_x/D, D_y/D, D_z/D], dtype='float') # solve the system
2 print(rf"X = {X} using Cramer's rule.")
3 # X = [1.40570175 0.68640351 2.95614035] using Cramer's rule.
```

Solution to #3

numpy also has a function to solve linear systems directly. It uses LU decomposition at its base.

```
1 X = np.linalg.solve(A, B)
2 print(rf"X = {X} using numpy's own solver.")
3 # X = [1.40570175 0.68640351 2.95614035] using numpy's own solver.
```

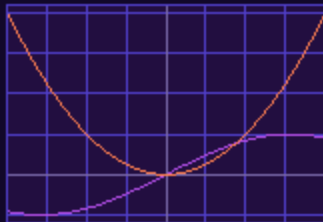
Part 3

The `scipy`, `sympy` and `matplotlib`
libraries

Numerical and symbolic computing

Example 2

1. (MMC/2013) How many real solutions are there in the equation $\sin(x) = x^2$?
2. Find the area between the curves $y = \sin(x)$ and $y = x^2$.



Solution outline

1. Determine the number of solutions graphically.
2. Find the exact values of the solutions.
3. Calculate the area between the curves using any of the four methods.
 - * Using trapezoidal rule
 - * Using Simpson's one-third rule
 - * Using Gaussian quadrature
 - * Using symbolic integration

The matplotlib library

Solution to #1

We use `matplotlib.pyplot` to graph the left and right sides of the equation and visually count the number of intersections.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

Generate a numpy array of 200 items of equal spacing from $-\pi$ to π .

```
1 x = np.linspace(-np.pi, np.pi, 200)
```

Create a figure and axis.

```
1 fig, ax = plt.subplots()
```

Plot the functions.

```
1 ax.plot(x, np.sin(x), label=r"$f(x) = \sin x$")
2 ax.plot(x, x**2, label=r"$g(x) = x^2$")
```

The matplotlib library

Solution to #1 (continued)

Add grid and axis lines.

```
1 ax.axhline(0, color='black')
2 ax.axvline(0, color='black')
```

Set labels and title.

```
1 ax.set_xlabel(rf"$x$")
2 ax.set_ylabel(rf"$y$")
3 ax.set_title(rf"Solutions to the equation $\sin x = x^2$")
```

Add legend and grid.

```
1 ax.legend()
2 ax.grid()
```

Show the plot. Note: Not required in a Jupyter notebook.

```
1 plt.show()
```

The scipy library

Solution to #1 (continued)

scipy's `scipy.optimize.fsolve()` function is a wrapper to a Fortran subroutine that uses a modification of Powell hybrid method to solve nonlinear equations.

```
1 import numpy as np
2 from scipy.optimize import fsolve
```

Write the equation as a function $f(x) = \text{left side} - \text{right side} = 0$.

```
1 eqn = lambda x : np.sin(x) - xxx2
```

Solve the equation. Multiple guess values can be fed to find multiple solutions.

```
1 roots = fsolve(eqn, [-0.05, 1])
2 print(roots)      # [4.50822753e-14  8.76726215e-01]
```

The scipy library

Solution to #2

scipy's `scipy.integrate` contains many methods for numerical integration.

* trapezoidal rule: $I = (h/2)(y_0 + 2y_1 + \dots + 2y_{(n-1)} + y_n)$

* Simpson's one-third rule: $I = (h/3)(y_0 + 4y_1 + 2y_2 + \dots + 2y_{(n-2)} + 4y_{(n-1)} + y_n)$

```
1 import numpy as np
2 from scipy.integrate import trapezoid, simpson
```

Divide the region into 100 points.

```
1 x = np.linspace(roots[0], roots[1], 100)
2 y = eqn(x)      # Calculate f_upper - f_lower = sin(x) - x^2
```

Calculate the integral.

```
1 print(rf"A = {trapezoid(y, x)} by trapezoidal rule.")    # 0.13568369268723793
2 print(rf"A = {simpson(y, x)} by Simpson's rule.")      # 0.13569750707724645
```

The scipy library

Solution to #2 (continued)

scipy's `scipy.integrate.quad()` function uses 21-point Gauss-Kronrod quadrature to numerically evaluate integrals.

```
1 | from scipy.integrate import quad
```

`quad` requires three arguments: the integrand, and the lower and upper bounds of the integral. It returns two items: the value of the integral, and the error parameters.

```
1 | # *iterable unpacks a list or a tuple into comma-separated items.  
2 | # In the below example, *roots = *[roots[0], roots[1]] = roots[0], roots[1].  
3 | print(rf"A = {quad(eqn, *roots)[0]} by scipy quadrature.")    # 0.1356975072306028
```

The sympy library

Solution to #2 (continued)

sympy is a library used for symbolic mathematics, similar to Mathematica or Maple.

```
1 | import sympy as sp
```

Specify the symbolic variable.

```
1 | x = sp.symbols("x")
```

Construct the equation or function based on that symbolic equation.

```
1 | eqn = sp.sin(x) - x**2
```

sympy also has `solve()` function for symbolic solutions of equations, and `nsolve()` for numerical solutions.

```
1 | root_1 = sp.nsolve(eqn, x, 0)
2 | root_2 = sp.nsolve(eqn, x, 1)
```

The sympy library

Solution to #2 (continued)

Symbolically evaluate the definite integral.

```
1 print(rf"A = {sp.integrate(eqn, (x, root_1, root_2))} by symbolic integration.")
2 # 0.135697507230603
```


Part 4

The pandas library

The pandas library

Example 3

Given the population of the Philippines from 1840 to 2024 based on national census, answer the following questions.

1. The population growth of a certain country is said to be *sustainable* if the annual growth rate is between 0.5% and 1.2%. List out the years in which the population growth of the Philippines was sustainable.
2. Based from the provided population data, calculate the average annual growth rate in percent for each listed year.
3. Based from the provided annual growth rate, calculate the maximum and minimum growth rate, and the respective years in which they occurred.

The pandas library

pandas is a Python library used for data analysis.

```
1 | import pandas as pd
```

pandas can import Excel and CSV files.

```
1 | ph_population = pd.read_excel("2025_T1_4.xlsx")
```

Imported data can sometimes contain invalid data like `NaN` (not a number) or `None` values. `dropna()` can be used to remove these values.

```
1 | ph_population = ph_population.dropna()
```

Like lists or *numpy* arrays, we can specify the rows that we will get as data.

```
1 | ph_population = ph_population[1:]    # Exclude the first row (i.e. index 0).
```

`.columns` can be used to specify the column names.

```
1 | ph_population.columns = ["Year", "Population", "Annual Growth Rate (%)", "Source"]
```

The pandas library

Solution to #1

We can filter the data by specifying conditions within square brackets.

```
1 sustainable = ph_population[ph_population["Annual Growth Rate (%)"] >= 0.5]
2 sustainable = ph_population[ph_population["Annual Growth Rate (%)"] <= 1.2]
```

We can also specify which columns will be considered.

```
1 print(sustainable[["Year", "Annual Growth Rate (%)"]])
```

Year	Annual Growth Rate (%)
1870	0.78
1887	0.72
1896	0.5
2024	0.8

The pandas library

Solution to #2

`.diff()` calculates the difference between consecutive values.

```
1 ph_population["Annual Increase"] = ph_population["Population"].diff()/ \
2                               ph_population["Year"].diff()
```

`.shift()` shifts the values in a column by a specified number of periods.

```
1 ph_population["Calculated Rate (%)"] = ph_population["Annual Increase"]/ \
2                               ph_population["Population"].shift(1) * 100
```

Show only the last two rows.

```
1 print(ph_population[["Year", "Population", "Annual Increase",
2                               "Calculated Rate (%)"]][-2:])
```

Year	Population	Annual Increase	Calculated Rate (%)
2020	109033245	1610788.4	1.595167
2024	112729484	924059.75	0.847503

The pandas library

Solution to #3

Some statistical measures that can be computed using *pandas* are `max()`, `min()`, `mean()`, `median()`, `std()`, `count()` and `sum()`.

```
1 # Maximum and minimum annual growth rates.
2 print(ph_population["Annual Growth Rate (%)"].max())    # 3.08
3 print(ph_population["Annual Growth Rate (%)"].min())    # 0.5
```

`.idxmax()` and `.idxmin()` returns the index numbers of the maximum and minimum values, respectively.

Using that index, we can retrieve the corresponding row using `.loc[]`.

```
1 # Year with which they respectively occurred.
2 print(ph_population.loc[ph_population["Annual Growth Rate (%)"].idxmax(), "Year"]) # 1970
3 print(ph_population.loc[ph_population["Annual Growth Rate (%)"].idxmin(), "Year"]) # 1896
```

The pandas library

EXTRA: From tables to graphs

`.to_numpy()` converts a pandas Series to a *numpy* array.

```
1 import matplotlib.pyplot as plt
2
3 year = ph_population["Year"].to_numpy(dtype='float')
4 population = ph_population["Population"].to_numpy(dtype='float')
```

From this, we can plot the data using *matplotlib*.

```
1 # A vertical bar plot
2 fig, ax = plt.subplots()
3 ax.bar([str(y) for y in year], population) # Convert years from numbers to strings
4 ax.tick_params("x", rotation=90)          # Rotate x-axis labels by 90 degrees
5 ax.set_xlabel(ph_population.columns[0])    # Set x-axis label
6 ax.set_ylabel(ph_population.columns[1])    # Set y-axis label
7 ax.set_title("Philippine Population (PSA Census Data)") # Set plot title
```

The pandas library

EXTRA: From tables to graphs

Several plots can be placed in a single figure using `plt.subplots()`.

```
1 fig, ax = plt.subplots(1, 4)                # Set up 4 subplots in a row
2 fig.set_size_inches(24, 6)                  # Set the overall figure size
3
4 for i, cur_col in enumerate((1, 2, 4, 5)) : # Loop through the columns we want to plot
5     # Plot the data
6     ax[i].plot(year, ph_population[ph_population.columns[cur_col]], 'ro-', linewidth=2)
7
8     # Set the axis labels
9     ax[i].set_xlabel(ph_population.columns[0])
10    ax[i].set_ylabel(ph_population.columns[cur_col])
11
12    # Set the title
13    ax[i].set_title(ph_population.columns[cur_col])
```


The pandas library

EXTRA: Regression analysis

`scipy.optimize.curve_fit()` function can be used to construct regression models from given data.

```
1 from scipy.optimize import curve_fit
2
3 lin_curve = lambda x, A, B : A + B * x          # linear regression equation  $y = A + Bx$ 
4 pow_curve = lambda x, A, B : A * B ** x        # power regression equation  $y = A * B^x$ 
5
6 lin_params, _ = curve_fit(lin_curve, year, population)  # fit linear curve to data
7 pow_params, _ = curve_fit(pow_curve, year, population) # fit power curve to data
8
9 # Plot the data and the resulting regression curves.
10 fig, ax = plt.subplots()
11 ax.scatter(year, population)
12 ax.plot(year, lin_curve(year, *lin_params), linestyle='dashed', color='red',
13         label=rf"population = {lin_params[0]:.2f} + {lin_params[1]:.2f} * year")
14 ax.plot(year, pow_curve(year, *pow_params), linestyle='dashed', color='green',
15         label=rf"population = {pow_params[0]:.2f} * {pow_params[1]:.2f}^x")
16 ax.set_xlabel("Year")
17 ax.set_ylabel("Population")
18 ax.legend()
```

The pandas library

EXTRA: Regression analysis

`scipy.optimize.curve_fit()` does not have a direct function to calculate R^2 values. So, we have to construct it ourselves.

```
1 # Calculate R^2 value
2 def r_squared(y_actual, y_calculated) :
3     residuals = y_actual - y_calculated
4     ss_res = np.sum(residuals**2)
5     ss_tot = np.sum((y_actual - np.mean(y_actual))**2)
6
7     return 1 - ss_res/ss_tot
8
9 # Linear curve
10 r_squared_linear = r_squared(population, lin_curve(year, *lin_params))
11 print(f"R^2 = {r_squared_linear:.2f} for linear curve")
12
13 # Power curve
14 r_squared_power = r_squared(population, pow_curve(year, *pow_params))
15 print(f"R^2 = {r_squared_power:.2f} for power curve")
```

An alternative to *scipy* for regression analysis is *scikit-learn*.

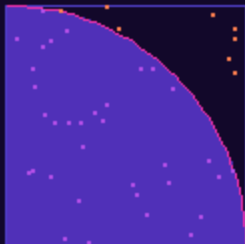
Part 4

**** A famous math example ****

A famous math example

Example 4

Random points (x, y) are placed in the Cartesian plane such that $0 \leq x, y \leq 1$. Estimate the value of π .



Solution

$$* A(\text{square}) = 1^2 = 1$$

$$* A(\text{quarter-circle}) = (1/4)(\pi * 1^2) \\ = \pi/4$$

$$* A(\text{quarter-circle}) / A(\text{square}) = \\ (\pi/4) / 1 = \pi/4$$

$$* \pi = 4 * A(\text{quarter-circle}) / \\ A(\text{square}) \\ = 4 * N(\text{points in quarter-circle}) / \\ N(\text{points in square})$$

Part 5

Next steps

Next steps

1. Create your Github account.
2. Read the library documentation **a lot**.
3. Stop coding examples. Start building your own projects.
4. Feel free to make mistakes.
5. Feel free to ask questions.
6. When using AI, **always** verify the results.
7. Find your desired specialization (e.g. actuarial science, finance, AI/ML, software and game development, etc).
8. Join a community (e.g. Reddit, Discord, community groups, etc).
9. Continue learning.

Thank you very much!

Do you have any questions?